Vanish

Self-destructing secret notes, containerised with Docker and deployed on Amazon EC2

Member A · Member B · Member C

November 2026

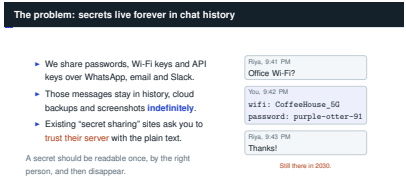
Cloud Computing Project, Your College Name

[A] Hi, we are Member A, Member B and Member C. Our cloud computing project is Vanish: a web app for sharing a secret that deletes itself. We packaged it with Docker and deployed it on an Amazon EC2 virtual machine. In the next ten minutes we'll cover the problem, the design, a demo, how we deployed it, and what we learned.

Agenda

1. The problem and our objectives
2. What Vanish does: a short demo
3. Architecture on Amazon EC2
4. Implementation: encryption, backend
5. Docker, Docker Compose and Caddy
6. Building and deploying to EC2
7. Testing, security and cost
8. Lessons learned and future work

[A] Here is our agenda. First the problem and our objectives, then what the app does with real screenshots. Member C will explain the architecture on EC2, Member A and Member B the implementation, and then Member C shows how we containerised and deployed it. We finish with testing, security, cost, and lessons learned.

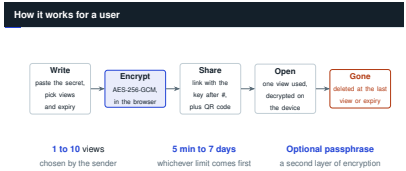


[A] Think about the last time you sent someone a password. It's probably still sitting in that chat, in cloud backups, maybe in a screenshot. Messages last forever, but secrets shouldn't. And the websites that offer one-time links usually receive your secret in plain text, so you have to trust them completely.

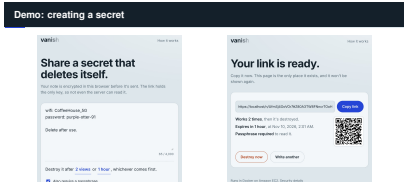
Objectives

1. **Self-destruct**: a note expires after *N* views or *T* time, whichever comes first.
2. **Zero knowledge**: the server stores only ciphertext and can never read a note.
3. **Correct under load**: two people can never both get the last view.
4. **Containerised**: the whole app runs from Docker images with one command.
5. **Deployed on a public cloud**: one Amazon EC2 instance, served over HTTPS.

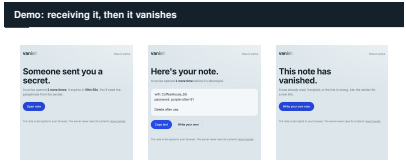
[A] We set five objectives. Notes must self-destruct after a number of views or a time limit. The server must never be able to read them. The view limit must hold even when two people open the link at the same moment. The app must be fully containerised. And it must run on a public cloud, Amazon EC2, over HTTPS.



[B] Here's the flow. You write a secret and choose a view limit and an expiry. Your browser encrypts it with AES-256 before anything is sent. You get a link and a QR code. When the recipient opens it, one view is used and the note is decrypted on their device. At the last view, or when time runs out, it's gone for good. You can also add a passphrase as a second lock.



[B] This is the app. I paste our café Wi-Fi password, choose two views or one hour, and add the passphrase “mango”. When I press Create secret link, the text dissolves: it’s being encrypted right here in the browser. On the right is the result: the link, a QR code for phones, and a summary of the rules. Only scrambled bytes were sent to the server.

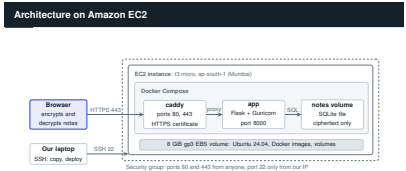


Countdown; opening the page uses no view

Decrypted on the recipient's device

After the last view: nothing left

[B] The recipient sees that a secret is waiting, how many views remain and a live countdown. Loading this page doesn't use a view, so chat apps that preview links can't burn it by accident. After entering the passphrase, the note is decrypted on their device. Once the last view is used, the server deletes it, and anyone opening the link again sees "This note has vanished."



[C] Everything runs on one Amazon EC2 virtual machine: a t3.micro running Ubuntu in the Mumbai region. A security group, AWS's firewall, lets anyone in on ports 80 and 443, but SSH only from our own IP. Inside, Docker Compose runs two containers. Caddy is the public web server: it gets a free HTTPS certificate and forwards requests to our app container, Flask with Gunicorn. The app keeps notes in a SQLite file on a Docker volume, which lives on the instance's EBS disk.

Cloud services and features used

Service / feature	How Vanish uses it	Setting
EC2 instance	Runs Docker and both containers	t3.micro, 2 vCPU, 1 GiB
AMI	Operating system image	Ubuntu Server 24.04 LTS
User data	Installs Docker and swap on first boot	`deploy/user-data.sh`
Instance metadata	`deploy.sh` looks up the public IP (IMDSv2)	169.254.169.254
Security group	Stateful firewall in front of the instance	22 (our IP), 80, 443
Key pair	Password-less SSH login	RSA, `.pem`
EBS volume	Disk for OS, images and the notes database	8 GiB gp3
Public IPv4	Address users connect to; HTTPS certificate is for it	auto-assigned
EC2 monitoring	CPU, network and status checks	Monitoring tab

[C] These are the EC2 features we used. The instance type and the Ubuntu AMI. A user-data script that installs Docker automatically at first boot. The instance metadata service, which our deploy script queries for the public IP. A security group as the firewall, a key pair for SSH, an 8 GiB EBS volume for storage, the public IPv4 address, and EC2's built-in monitoring.



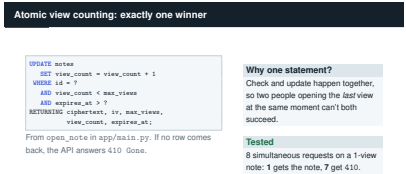
[B] This is the core idea. The part of the link before the hash tells the server which note to fetch. The part after the hash is the 256-bit decryption key, and browsers never send that part to any server. So our EC2 instance only ever stores scrambled ciphertext. Encryption uses AES-256-GCM from the browser's built-in Web Crypto API, and the optional passphrase adds a second layer derived with PBKDF2.

The API: five small endpoints

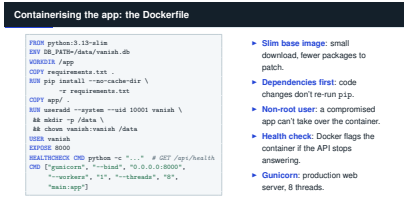
Method	Path	What it does
`POST`	`/api/notes`	Store `ciphertext`, `iv`, view limit (1–10), expiry (5 min–7 days)
`GET`	`/api/notes/{id}`	Views left and expiry; **does not use a view** (safe for link previews)
`POST`	`/api/notes/{id}/open`	Atomically use one view and return the ciphertext; `410 Gone` afterwards
`DELETE`	`/api/notes/{id}`	Destroy early with the secret delete token (stored as a SHA-256 hash)
`GET`	`/api/health`	Liveness check used by the Docker health check

A background thread deletes expired notes every 60 seconds; every read also checks the expiry time.

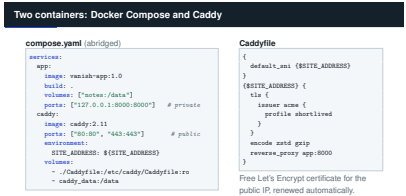
[A] The backend is one Python file with five endpoints. Creating a note stores only ciphertext. Reading the metadata never uses a view, which is why WhatsApp previews can’t burn a note. Opening uses one view atomically. There’s an early-destroy endpoint protected by a secret token, and a health check that Docker uses. A background thread deletes expired notes every minute.



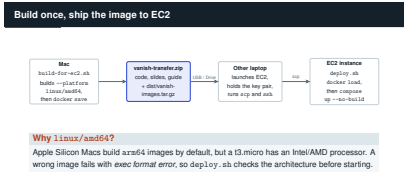
[A] The most important line of backend code is this single SQL statement. It increases the view count only if the note still has views left and hasn't expired, and returns the note in the same step. Because the check and the update are one atomic operation, two people can't both get the last view. Our test fires eight requests at the same moment at a one-view note: exactly one wins and seven get 410 Gone.



[C] This is our Dockerfile. It starts from the official slim Python image, installs Flask and Gunicorn first so that code changes rebuild quickly, copies the app, and runs it as a non-root user. A health check calls our API every 30 seconds, and Gunicorn serves the app with eight threads.



[C] Docker Compose runs two containers. The app's port is bound to localhost only, so the internet can reach it only through Caddy. Notes live in a named volume, so they survive restarts and rebuilds. On the right is the entire Caddyfile: it requests a free Let's Encrypt certificate for our IP address and forwards every request to the app. HTTPS isn't optional here: browsers only allow the Web Crypto API on secure pages.



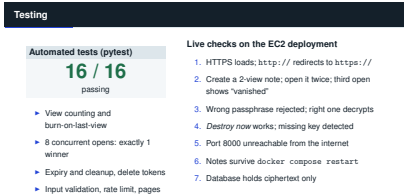
[C] We build the images once and ship them. On the Mac, one script builds the app image for linux/amd64, downloads Caddy, and saves both into a single archive inside the project zip. We carry that zip to the laptop we deploy from, copy it to EC2 with scp, and our deploy script loads the images and starts them, with no build on the server. The platform flag matters: Apple Silicon builds arm64 by default, which won't run on a t3.micro, so the deploy script checks this before starting.



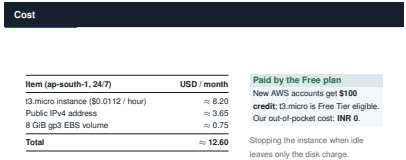
[C] Deployment takes four steps. We launch a t3.micro with Ubuntu, open HTTP and HTTPS, restrict SSH to our IP, and paste our user-data script, which installs Docker at first boot. Then we copy the zip over with scp, log in with ssh, unzip it and run one script. It finds the public IP from the instance metadata service, loads the images, starts both containers, and Caddy obtains the certificate within a minute.



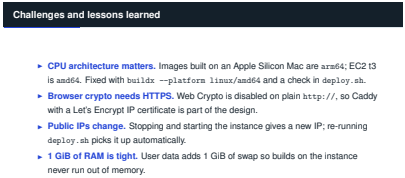
[B] Security works in two layers. For data: the server only ever sees ciphertext, the key lives in the URL fragment and is removed from the address bar after opening, delete tokens are hashed, and note creation is rate-limited. For infrastructure: the security group exposes only three ports, the app container isn't directly reachable, it runs as a non-root user, and every response is served over HTTPS with strict browser security headers.



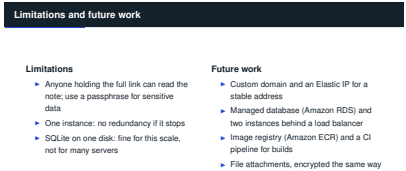
[A] We tested at two levels. Sixteen automated tests run against the real Flask app and all pass, including the concurrency test. Then on the live EC2 instance we checked HTTPS and the redirect, the full view and burn cycle, the passphrase, early destroy, that port 8000 is closed to the internet, that notes survive a restart, and that the database only contains ciphertext.



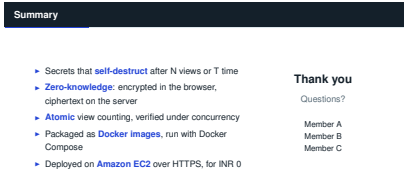
[C] Running all month, the instance, its public IP and its disk cost about twelve and a half dollars. New AWS accounts on the Free plan get a hundred dollars of credit and t3.micro is Free Tier eligible, so our out-of-pocket cost was zero. When we aren’t using it, we stop the instance, and only the small disk charge remains.



[B] We learned a few things the hard way. CPU architecture matters: images from an Apple Silicon Mac won't run on an Intel EC2 instance unless you build for linux/amd64. Browser encryption requires HTTPS, so getting a certificate without a domain became part of the design. Public IPs change when you stop an instance, so the deploy script re-detects it. And a one-gigabyte instance needs swap to build images safely.



[B] Some honest limits. Anyone with the full link can read the note, so sensitive notes should use a passphrase. One instance means no redundancy, and SQLite suits a single server. Next steps would be a custom domain with a stable Elastic IP, a managed database with two instances behind a load balancer, a container registry with automated builds, and encrypted file attachments.



[B] To sum up: Vanish lets you share a secret that destroys itself, it never sees your data in plain text, its view limits hold under concurrent access, and the whole thing is packaged with Docker and running on Amazon EC2 over HTTPS, at no cost to us. Thank you for watching.